

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2 «Реализация микросервисов»

Выполнила:

Клапнева Диана

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

Опираясь на документ, сформированный в рамках ДЗ4, реализуйте разделение вашего бэкенд-приложения на микросервисы.

Необходимо сделать отчёт по шаблону

Ход работы

Шаг 1: Вся суть работы - в разделении монолитного приложения на отдельные сервисы. В моем случае можно выделить сервис чата, сервис управления записями о недвижимости, сервис обработки сделок и сервис управления учетными записями пользователей. Общий репозиторий с моделями, контроллерами и data transfer objects мы разделяем на 4 репозитория, в которых находятся только модели/контроллеры, относящиеся к данному сервису. В index.ts также прописываем только относящиеся к микросервису маршруты.

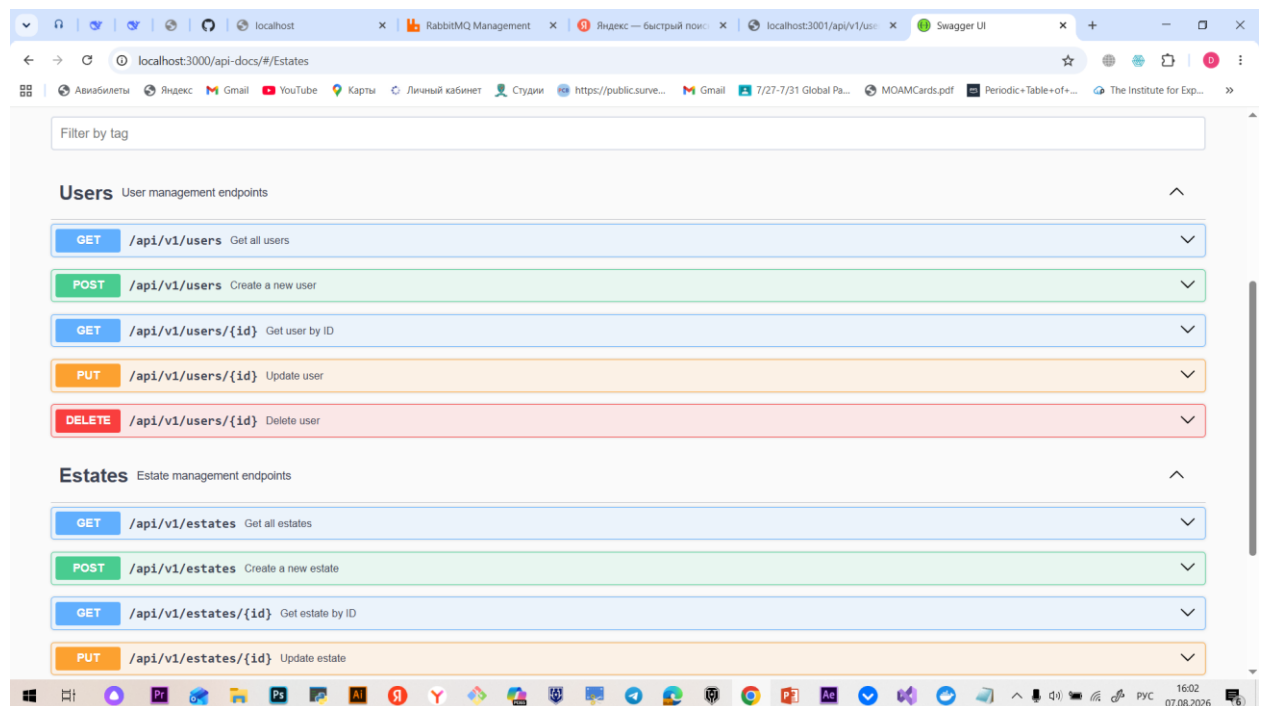
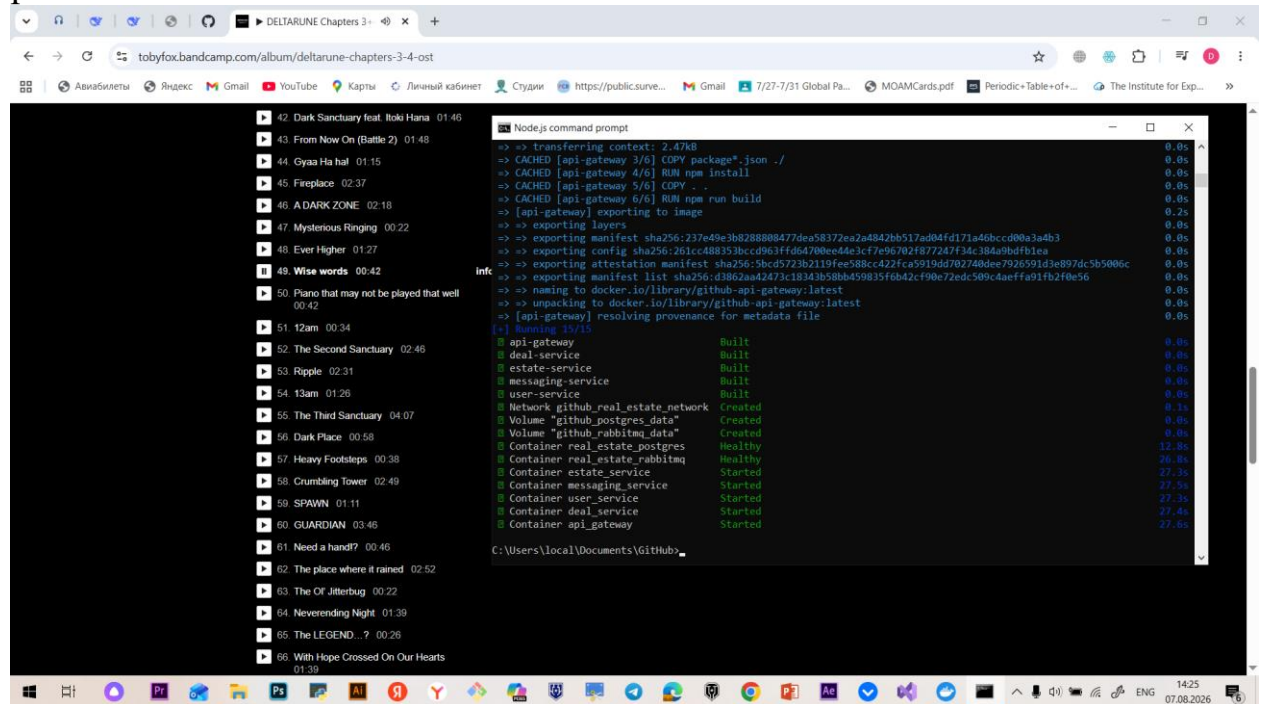
Рассмотрим это на примере микросервиса сделок – он у нас в репозитории deal-service. В папках controllers, models, dto, services лежат только deal.controller.ts, deal.entity.ts, deal.dto.ts и deal.service.ts соответственно.

Рассмотрим файлы src/index.ts и src/routes/index.ts:

src/index.ts включает в себя все запросы, связанные со сделками и все возможные http коды ответа. Все это в распознаваемом swagger формате.

По одному deal id можно найти сделку (варианты ответа 200 или 404 – нашли либо нет), изменить ее детали (200, 404 либо 422 – ошибка валидации), удалить ее (204 – удалено, 404). Создание новой сделки и получение всех сделок также

реализовано.



```
    '/{dealId}': {
      get: {
        summary: 'Get deal by ID',
        parameters: [
          { name: 'dealId', in: 'path', required: true, schema: { type: 'integer' } }
        ],
        responses: {
          '200': { description: 'Deal found' },
          '404': { description: 'Deal not found' }
        }
      },
      patch: {
        summary: 'Update deal',
        parameters: [
          { name: 'dealId', in: 'path', required: true, schema: { type: 'integer' } }
        ],
        requestBody: {
          content: {
            'application/json': {
              schema: {
                type: 'object',
                properties: {
                  landlord_id: { type: 'integer' },
                  tenant_id: { type: 'integer' },
                  period: { type: 'string' },
                  deal_status: { type: 'string', enum: ['pending', 'active', 'closed'] },
                  estate_ids: { type: 'array', items: { type: 'integer' } },
                  is_published: { type: 'boolean' }
                }
              }
            }
          }
        },
        responses: {
          '200': { description: 'Deal updated' },
          '404': { description: 'Deal not found' },
          '422': { description: 'Validation error' }
        }
      }
    },
  },
}
```

```
import { Router } from 'express';
import { DealController } from '../controllers/deal.controller';

const router = Router();
const dealController = new DealController();

router.get('/', dealController.listDeals);
router.get('/:dealId', dealController.getDeal);
router.post('/', dealController.createDeal);
router.patch('/:dealId', dealController.updateDeal);
router.delete('/:dealId', dealController.deleteDeal);

export default router;
```

Шаг 2: создадим репозиторий api-service для маршрутизации. Здесь мы используем http-proxy-middleware чтобы создать прокси. Прокси нужен, чтобы иметь доступ ко всем микросервисам согласно созданной маршрутизации, также тут будет логирование запросов ко всем сервисам, что лучше для отслеживания операций. Фронтенд не должен знать адреса всех серверов, а только шлюз — для безопасности. Также по новому маршруту можно легче создать новую рабочую версию api.

```
import express, { Request, Response, NextFunction } from 'express';
import cors from 'cors';
import helmet from 'helmet';
import http from 'http';
import https from 'https';
import dotenv from 'dotenv';

dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

app.use(helmet());
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// это логирование запросов
app.use((req: Request, res: Response, next: NextFunction) => {
  console.log(`[Gateway] ${req.method} ${req.url}`);
  next();
});

app.get('/health', (req: Request, res: Response) => {
  res.json({
    service: 'api-gateway',
    status: 'OK',
    timestamp: new Date().toISOString(),
  });
});
```

Ниже часть кода для создания прокси-запроса к микросервисам.

```
app.use('/api/v1/users', (req, res) => {
  proxyRequest(req, res, 'http://localhost:3001', 'User Service');
});

app.use('/api/v1/estates', (req, res) => {
  proxyRequest(req, res, 'http://localhost:3002', 'Estate Service');
});

app.use('/api/v1/deals', (req, res) => {
  proxyRequest(req, res, 'http://localhost:3003', 'Deal Service');
});

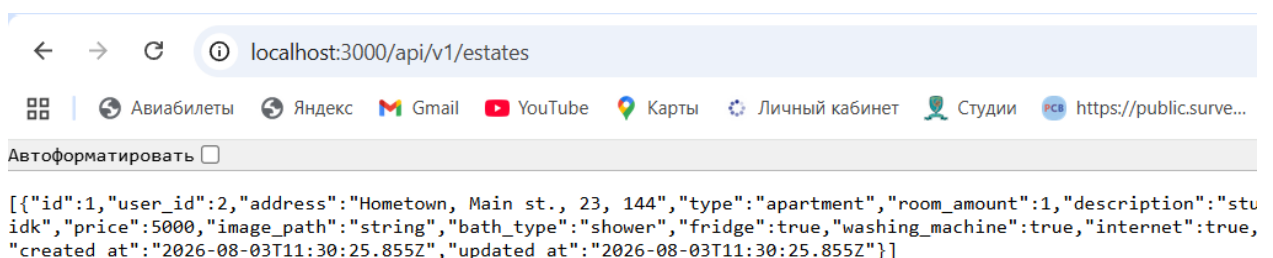
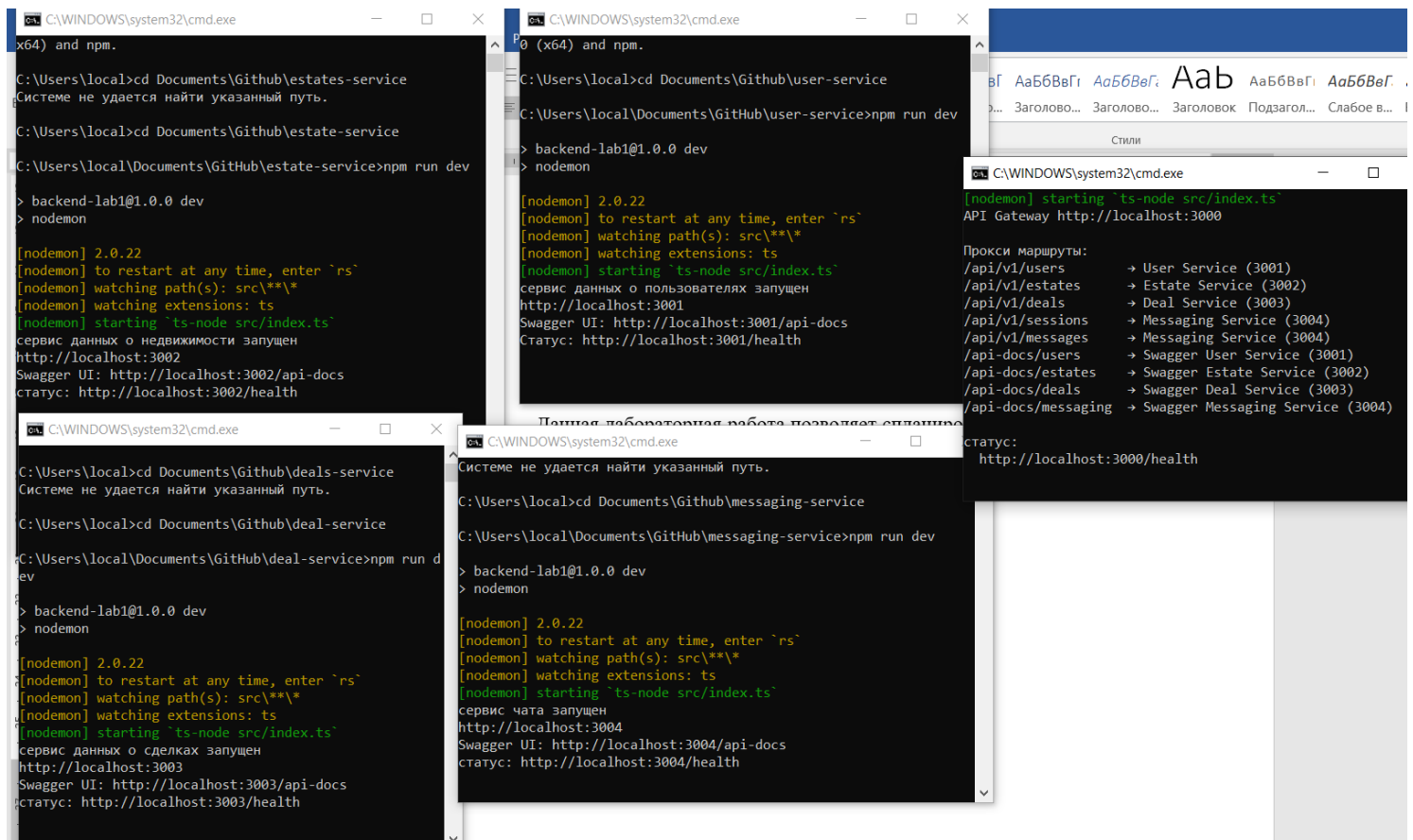
app.use('/api/v1/sessions', (req, res) => {
  proxyRequest(req, res, 'http://localhost:3004', 'Messaging Service');
});

app.use('/api/v1/messages', (req, res) => {
  proxyRequest(req, res, 'http://localhost:3004', 'Messaging Service');
});
```

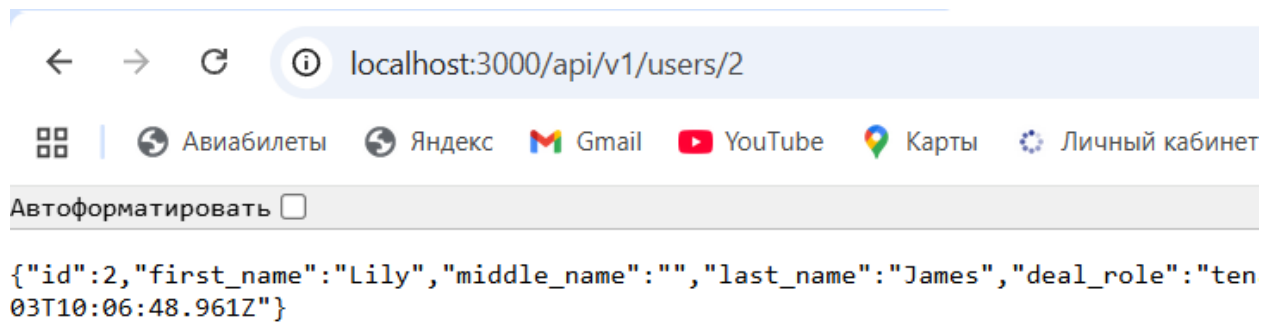
У нас есть функция проксирования запросов к серверу. Мы отправляем запрос на порт 3000, а прокси выбирает, на какой порт его перенаправить. Алгоритм такой:

1. Клиентский запрос через шлюз
2. Логирование запроса
3. Определение, к какому сервису запрос
4. Создание нового запроса к микросервису
5. Микросервис принимает запрос от прокси, обрабатывает его и возвращает ответ
6. API шлюз возвращает ответ клиенту

Шаг 3: Запустим все микросервисы и посмотрим на работу документации в Swagger UI и запросов через шлюз.



Запрос к сервису недвижимости через шлюз работает успешно.



Запрос поиска пользователя по id тоже успешен.

Вывод

Данная лабораторная работа помогла мне на практике понять, что такое прокси и как он действует, зачем нужно разбиение приложения на микросервисы и как централизовать отправку запросов к ним.